Annotated Lean Reference Specifications

Companion to
Leakage-Resilient Computation via Matrix Refreshing

1 Reading and checking the specifications

This note accompanies *Leakage-Resilient Computation via Matrix Refreshing*. All source paths below are relative to `version-1.0/` in the MatrixRefresh repository. Lean checks that a proof establishes its formal statement, while judging whether that statement expresses the intended claim remains for human readers. To support that judgment, we print selected data interfaces, component proposition declarations, and reference theorem statements in `lean/Refreshing.lean`, `lean/Compiler.lean`, and `lean/EfficientSimulation.lean`. Mathematical explanations appear between logical blocks of code. These interfaces cover binary continual refreshing, the NAND compiler, its counted simulator, and repeated stateful evaluation; the word `Direct` in Lean paths and names refers to the NAND compiler.

The listings are generated directly from those files, omitting comments, redundant blank lines, and file setup commands such as imports, namespace and section wrappers, variables, open commands, and options. Each public file contains its model, admitted construction certificates where needed, and one admitted main theorem. Refreshing states its components directly in its main theorem. Compiler and EfficientSimulation give their components named propositions, with numerical bounds written in those propositions, and each main theorem bundles them. For example, the compiler admits the proof-valued definition `raw_acceptance_positive`, which certifies that its raw rejection sampler can terminate; the sampler definition remains explicit in `lean/Compiler.lean`. The compiler file specifies source circuits, their compilation, physical execution, and causal public interaction; the simulation file contains the counted simulators. Probability, arithmetic, and sampler definitions are placed in the earliest public file that needs them; lower-level constructions are omitted from the printed listings.

Nontrivial construction certificates in the public files use `by sorry`, and their complete proofs are in `lean/Proofs/`. Elementary arithmetic and finite-index proofs may appear directly inside the public definitions. Proof-only definitions and theorem statements are absent from the public files. Neither the public compiler and simulation specifications nor their dependencies import that proof tree.

The displayed interfaces omit supporting proofs and lower-level implementation details; the complete Lean sources are available in the repository. The listings reproduce selected complete declarations from the public files. Unicode identifiers and operators are typeset with mathematical glyphs; an arrow at the start of a listing line indicates a wrapped source line.

Scope of the formal statements. The checked headline claims use deterministic strategies that adapt between leakage phases; the two parties' functions within a phase are chosen before either current answer is known. The security guarantees use the binary field, although several compiler helpers are field-parametric. The paper's averaging argument for randomized adversaries, its conversion from other Boolean gate bases to NAND, capped simulation, within-phase adaptive leakage, and the larger-field extension are outside these checked statements. Protocol randomness,

all rejected proposals and retries, causal public inputs, and authorized plaintext outputs are part of the models. The results concern mathematical probability distributions and a specified work counter; they do not certify extracted machine code.

The bundled reference theorem declarations end with `sorry`; they specify the claims that separate proof modules establish. The proof modules are:

- Continual refreshing: `Proofs.Refreshing.Model` and `Proofs.Refreshing.Theorem41`.
- Signed composition: `Proofs.Compiler.DirectComposition.Theorem57` for fixed labels and `Proofs.Compiler.DirectComposition.AdaptiveTheorem57` for causal history-dependent labels.
- Privacy of the compiler: `Proofs.Compiler.Direct.Theorem52`.
- Simulation for the compiler: `Proofs.EfficientSimulaiton.Direct.Theorem52`.
- Stateful privacy and simulation: `Proofs.Compiler.Stateful.Theorem61` and `Proofs.EfficientSimulaiton.Stateful.Theorem61`.

Each public module has exactly one admitted main theorem. The independent proof modules use model copies that omit these theorem admissions and prove the construction certificates. The generic probability and accounting definitions used by the counted simulator are included below. The reference statements and their proved counterparts are loaded through `Proofs.ReferenceClaims` and `Proofs.ProvedClaims` in separate Lean environments, so admitting a reference statement is not an assumption available to its proof. To build the proof modules and compare them with the reference statements, run the following commands from the `lean/` directory:

```
lake build
lake comparator --paranoid
```

The comparator checks the main theorem statements and construction certificates, compares their underlying definitions, audits proof dependencies, and checks the proof terms; `--paranoid` runs all six bundled checking backends. The option `--inadvisably-no-sandbox` disables the comparator's `bubblewrap` sandbox: project configuration, build scripts, and other executed code then run without this isolation, with the permissions of the invoking user. Untrusted code could access or modify files and compromise the audit result. Running with this option is at the reader's own risk. The displayed command uses the sandbox, which we recommend wherever supported. Only when an explicit unsandboxed fallback is necessary should the reader add `--inadvisably-no-sandbox`; this fallback does not provide the sandbox's isolation.

2 The refreshing model: `lean/Refreshing.lean`

Finite uniform probability. For a nonempty finite set S, the uniform probability mass function is

$$\mathrm{Unif}(S)(x) = \frac{1}{|S|}, \qquad \sum_{x \in S} \mathrm{Unif}(S)(x) = 1.$$

Lean's `PMF` uses nonnegative extended-real masses; the normalization proof is part of the construction in `transcriptDistribution` below.

Shares, states, and secret fibers. All share arithmetic is over $\mathbb{F}_2 = \mathbb{Z}/2\mathbb{Z}$. Write $[n] := \{i \in \mathbb{N} : i < n\}$, the set represented by `Fin n`. The abbreviations in the next block have the following mathematical meanings:

$$\begin{aligned} \mathsf{Bit} &:= \mathbb{F}_2, \\ \mathsf{Vector}(n) &:= \mathbb{F}_2^{[n]} = \mathbb{F}_2^n, \\ \mathsf{Share}(n) &:= \{v \in \mathsf{Vector}(n) : v \neq 0\} = U_n, \\ \mathsf{State}(n) &:= \mathsf{Share}(n) \times \mathsf{Share}(n) = U_n \times U_n, \\ \mathsf{RefreshMatrix}(n) &:= \mathrm{GL}_n(\mathbb{F}_2) = G_n. \end{aligned}$$

The same vector type represents rows and columns; their position in a matrix product determines the orientation. For $x = (\ell, r)$, the secret and its fiber are

$$\begin{aligned} \mathsf{secret}(x) &:= \ell \cdot r = \sum_{k=0}^{n-1} \ell_k r_k, \\ \mathsf{Encoding}(n, s) &:= \{x \in \mathsf{State}(n) : \mathsf{secret}(x) = s\} = \Omega_s. \end{aligned}$$

The final instance records the finiteness of the invertible-matrix group.

```
abbrev Bit := ZMod 2

abbrev Vector (n : ℕ) := Fin n → Bit

abbrev Share (n : ℕ) := {v : Vector n // v ≠ 0}

abbrev State (n : ℕ) := Share n × Share n

abbrev RefreshMatrix (n : ℕ) := (Matrix (Fin n) (Fin n) Bit)ˣ

def secret {n : ℕ} (x : State n) : Bit := x.1.val ▪ᵥ x.2.val

abbrev Encoding (n : ℕ) (s : Bit) := {x : State n // secret x = s}

instance (n : ℕ) : Fintype (RefreshMatrix n) := Fintype.ofFinite _
```

The refresh map. Given $M \in G_n$, the parties replace their shares by

$$\ell' := \ell \cdot M, \qquad r' := M^{-1} \cdot r.$$

The independent construction proofs show that the new shares are nonzero: multiplying a hypothetical zero output by the inverse map would make the old share zero. The resulting state has the same secret, since

$$(\ell \cdot M) \cdot (M^{-1} \cdot r) = \ell \cdot r.$$

```
def refreshLeft {n : ℕ} (l : Share n) (M : RefreshMatrix n) : Share n :=
  ⟨l.val ᵥ* (M : Matrix _ _ _), ModelSupport.refreshLeft_ne_zero l.val l.property M⟩

def refreshRight {n : ℕ} (r : Share n) (M : RefreshMatrix n) : Share n :=
  ⟨(↑(M⁻¹) : Matrix _ _ _) *ᵥ r.val, ModelSupport.refreshRight_ne_zero r.val r.property M⟩

def refresh {n : ℕ} (x : State n) (M : RefreshMatrix n) : State n :=
  (refreshLeft x.1 M, refreshRight x.2 M)
```

Initialization and fresh matrices. For $n \geq 2$, each fiber Ω_s is nonempty: with standard basis vectors e_0 and e_1, the pair $(e_0, s \cdot e_0 + e_1)$ encodes s and has nonzero shares. The first clause of `Refreshing.main_theorem` states this nonemptiness. The admitted `encodingWitness` certificate supplies the nonempty instance used by `transcriptDistribution`; independent proofs establish both the theorem clause and the certificate. Initialization samples $X_0 \leftarrow \mu_s := \mathrm{Unif}(\Omega_s)$, viewed as a distribution on all states, and each refresh matrix is distributed as $\mathrm{Unif}(G_n)$.

```
def encodingWitness {n : ℕ} (hn : 2 ≤ n) (s : Bit) :
    Nonempty (Encoding n s) := by
  sorry
```

Deterministic adaptive leakage. We identify Lean's `Bool` with $\{0,1\}$. The answer and history abbreviations are

$$\begin{aligned}
\mathsf{LeakageOutput}(\lambda) &:= \{0,1\}^{[\lambda]} = B_\lambda, \\
\mathsf{Answer}(\lambda) &:= B_\lambda \times B_\lambda = \mathcal{Z}_\lambda, \\
\mathsf{History}(\lambda, i) &:= \mathcal{Z}_\lambda^{[i]} = \mathcal{H}_i, \\
\mathsf{LeakageQuery}(n, \lambda) &:= (U_n \times G_n \to B_\lambda)^2 = \mathcal{Q}_{n,\lambda}, \\
\mathsf{Strategy}(n, T, \lambda) &:= \prod_{i \in [T]} (\mathcal{H}_i \to \mathcal{Q}_{n,\lambda}).
\end{aligned}$$

A deterministic strategy supplies, for every $0 \leq i < T$ and every $h \in \mathcal{H}_i$, a pair

$$A_i(h) = (f_{\mathsf{L}}, f_{\mathsf{R}}), \qquad f_{\mathsf{L}}, f_{\mathsf{R}} : U_n \times G_n \longrightarrow B_\lambda.$$

Both functions are fixed before either answer of the round is disclosed. Their joint answer on old state (ℓ, r) and matrix M is $(f_{\mathsf{L}}(\ell, M), f_{\mathsf{R}}(r, M))$. The strategy has no secret argument and is defined even at histories with probability zero.

```
abbrev LeakageOutput (leakage : ℕ) := Fin leakage → Bool

abbrev Answer (leakage : ℕ) := LeakageOutput leakage × LeakageOutput leakage

abbrev History (leakage i : ℕ) := Fin i → Answer leakage

abbrev LeakageQuery (n leakage : ℕ) :=
  (Share n → RefreshMatrix n → LeakageOutput leakage) ×
  (Share n → RefreshMatrix n → LeakageOutput leakage)

abbrev Strategy (n T leakage : ℕ) := (i : Fin T) → History leakage i.val → LeakageQuery n
  ↪ leakage

def answer {n leakage : ℕ} (f : LeakageQuery n leakage) (x : State n)
    (M : RefreshMatrix n) : Answer leakage := (f.1 x.1 M, f.2 x.2 M)
```

Chronological execution and its coins. Start from $(X_0, H_0) = (x_0, \varnothing)$. For $0 \leq i < T$, using the paper's round numbering,

$$\begin{aligned}
Z_{i+1} &:= \mathsf{answer}\big(A_i(H_i), X_i, M_{i+1}\big), \\
X_{i+1} &:= \mathsf{refresh}(X_i, M_{i+1}), \qquad H_{i+1} := H_i \| Z_{i+1}.
\end{aligned}$$

The coin space is

$$\mathsf{Coins}(n, T, s) := \mathsf{Encoding}(n, s) \times G_n^{[T]} = \Omega_s \times G_n^{[T]}.$$

Uniform sampling on that product gives an independent initial encoding and independent fresh matrices. The transcript map retains H_T, while both endpoint states remain private.

```
def runPrefix {n T leakage : ℕ} (A : Strategy n T leakage) (x₀ : State n)
    (matrices : Fin T → RefreshMatrix n) :
    (i : ℕ) → i ≤ T → State n × History leakage i
  | 0, _ => (x₀, Fin.elim0)
  | i + 1, hi =>
    let previous := runPrefix A x₀ matrices i (Nat.le_of_succ_le hi)
    let round : Fin T := ⟨i, Nat.lt_of_succ_le hi⟩
    let M := matrices round
    let z := answer (A round previous.2) previous.1 M
    (refresh previous.1 M, Fin.snoc previous.2 z)

abbrev Coins (n T : ℕ) (s : Bit) := Encoding n s × (Fin T → RefreshMatrix n)

def transcript {n T leakage : ℕ} (A : Strategy n T leakage) {s : Bit}
    (coins : Coins n T s) : History leakage T :=
  (runPrefix A coins.1.val coins.2 T le_rfl).2
```

Transcript distributions and distance. For a fixed strategy A, the distribution P_s^T is the push forward of the uniform coin distribution through the transcript map. For finite probability distributions, the distance used throughout the security statements is

$$\delta(P, Q) = \frac{1}{2} \sum_z |P(z) - Q(z)|.$$

The sum is an unnormalized mass sum, rather than a uniform average. The cardinalities used in the rate are $q = |U_n| = 2^n - 1$ and $K = |\mathcal{Z}_\lambda| = 2^{2\lambda}$.

```
def transcriptDistribution {n T leakage : ℕ} (hn : 2 ≤ n)
    (A : Strategy n T leakage) (s : Bit) : PMF (History leakage T) :=
  letI := encodingWitness hn s
  (PMF.ofFintype (fun _ : Coins n T s => (Fintype.card (Coins n T s) : ℝ≥0∞)⁻¹)
    (ModelSupport.uniform_total (Coins n T s))).map (transcript A)

def statisticalDistance {α : Type*} [Fintype α] (P Q : PMF α) : ℝ :=
  (1 / 2 : ℝ) * ∑ a, |(P a).toReal - (Q a).toReal|
```

3 The refreshing theorem: `lean/Refreshing.lean`

For readability, write the two real-valued expressions

$$B_1(C, n, T, \lambda) = C \cdot T \cdot K^3 \cdot q^{-1/9}, \qquad B_2(C', n, T, \lambda) = C' \cdot T \cdot 2^{6\lambda - n/9}.$$

The fractional exponents are real powers; in particular, `1 / 9` is interpreted in $\mathbb{R}$, not as natural-number division. The theorem writes these expressions out in full.

The quantifiers in the theorem express one pair of absolute constants $C \geq 1$ and $C' \geq 1$, uniform over $n \geq 2$, $T \geq 0$, $\lambda \geq 0$, and every deterministic adaptive strategy A, for which

$$\delta\left(P_0^T, P_1^T\right) \leq C \cdot T \cdot K^3 \cdot q^{-1/9} \leq C' \cdot T \cdot 2^{6\lambda - n/9}.$$

The two secret experiments use the same A. The separate proof implementation chooses $C = 12000$ and $C' = 24000$.

The public refreshing theorem joins encoding nonemptiness and the security bound.

```
theorem main_theorem :
    (∀ {n : ℕ} (hn : 2 ≤ n) (s : Bit), Nonempty (Encoding n s)) ∧
    (∃ C C' : ℝ, 1 ≤ C ∧ 1 ≤ C' ∧
      ∀ (n T leakage : ℕ) (hn : 2 ≤ n),
        C * (T : ℝ) * (((2 ^ (2 * leakage) : ℕ) : ℝ) ^ 3) *
            Real.rpow (((2 ^ n - 1 : ℕ) : ℝ)) (-(1 / 9 : ℝ)) ≤
          C' * (T : ℝ) *
            Real.rpow ((2 : ℕ) : ℝ) (((6 : ℕ) : ℝ) * (leakage : ℝ) - (n : ℝ) / 9) ∧
        (∀ A : Strategy n T leakage,
          statisticalDistance (transcriptDistribution hn A 0)
            (transcriptDistribution hn A 1) ≤
              C * (T : ℝ) * (((2 ^ (2 * leakage) : ℕ) : ℝ) ^ 3) *
                Real.rpow (((2 ^ n - 1 : ℕ) : ℝ)) (-(1 / 9 : ℝ)))) := by
  sorry
```

4 The NAND source and refresh chains of the compiler

The source interface is in `lean/Compiler.lean`. For all P, U, T, and k in $\mathbb{N}$, it records P private inputs, U fixed public inputs or constants, T NAND gates, and k designated output positions. A gate may use the same wire twice, and a wire may feed arbitrarily many later gates. The case $T = 0$ is allowed. The inductive type below makes the chronological condition explicit: both operands of a new gate name previously available wires.

```
inductive NandProgram (inputs : ℕ) : ℕ → Type where
  | nil : NandProgram inputs 0
  | snoc {gates : ℕ} (prior : NandProgram inputs gates)
      (left right : Fin (inputs+gates)) : NandProgram inputs (gates+1)

def eval {inputs : ℕ} (input : Fin inputs → Refreshing.Bit) :
    {gates : ℕ} → NandProgram inputs gates → Fin (inputs+gates) → Refreshing.Bit
  | 0, .nil => input
  | _+1, .snoc prior left right =>
      let old := prior.eval input
      Fin.snoc old (1+old left*old right)
```

The public source contains no private values. For private input $x \in \mathbb{F}_2^P$, the functions below assemble $(x, \mathsf{publicValues})$, evaluate every source wire, and select the authorized output tuple. The map $\mathsf{outputWires} : [k] \to [P + U + T]$ may repeat positions and may directly designate inputs.

```
structure Source (privateInputs publicInputs gates outputs : ℕ) where
  program : NandProgram (privateInputs+publicInputs) gates
  publicValues : Fin publicInputs → Refreshing.Bit
  outputWires : Fin outputs → Fin (privateInputs+publicInputs+gates)

def input (privateValues : Fin P → Refreshing.Bit) : Fin (P+U) → Refreshing.Bit :=
  Fin.addCases privateValues source.publicValues

def evaluate (privateValues : Fin P → Refreshing.Bit) : Fin (P+U+T) → Refreshing.Bit :=
  source.program.eval (source.input privateValues)

def outputValues (privateValues : Fin P → Refreshing.Bit) (j : Fin k) : Refreshing.Bit :=
  source.evaluate privateValues (source.outputWires j)
```

The executable compiler is in `lean/Compiler.lean`. A compilation prefix records the actual instruction program, the original location of every source wire, and its current refresh-chain endpoint. The trusted initial state contains precisely the P private input encodings.

```
structure Compilation (privateInputs publicInputs sourceGates : ℕ) where
  operations : ℕ
  program : Operations.Program Refreshing.Bit privateInputs operations
  count : operations=publicInputs+3*sourceGates
  original : Fin (privateInputs+publicInputs+sourceGates) → Fin (privateInputs+operations)
  live : Fin (privateInputs+publicInputs+sourceGates) → Fin (privateInputs+operations)
```

For each source NAND, the compiler refreshes its left operand's current endpoint, refreshes its right operand's updated endpoint, and computes NAND on the two fresh pairs. If the source operands coincide, the second refresh follows the first; the computation still receives two distinct encoding locations. The update below is the instruction construction used by the actual execution.

```
def step (state : Compilation P U T) (left right : Fin (P+U+T)) : Compilation P U (T+1) :=
  let N := P+state.operations
  let afterLeft : Fin (P+U+T) → Fin (N+1) :=
    fun w => if w=left then Fin.last N else (state.live w).castSucc
  let afterRight : Fin (P+U+T) → Fin (N+1+1) :=
    fun w => if w=right then Fin.last (N+1) else (afterLeft w).castSucc
  { operations := state.operations+3
    program := ((state.program.snoc (.refresh (state.live left))).snoc
      (.refresh (afterLeft right))).snoc
        (.compute ⟨.complement 1,(Fin.last N).castSucc,Fin.last (N+1)⟩)
    count := by have h := state.count; omega
    original := Fin.lastCases (Fin.last (N+1+1))
      (fun w => (state.original w).castSucc.castSucc.castSucc)
    live := Fin.lastCases (Fin.last (N+1+1)) (fun w => (afterRight w).castSucc) }
```

Public-input samplers precede these three-operation gate blocks. The proved count is $S = 3T + U$, including $T = 0$.

```
def compilation : Compilation P U T := source.program.compile source.publicValues

def operationCount : ℕ := source.compilation.operations
```

The compiler allocates $P + S$ persistent encoding locations. The trusted decoder uses the original source-wire map, so subsequent input-port refreshes neither change the designated output tuple nor disclose an encoding pair.

```
def compiledProgram : Operations.Program Refreshing.Bit P source.operationCount := source.
   ↪ compilation.program

def outputSlot (j : Fin k) : Fin (P+source.operationCount) :=
  source.compilation.original (source.outputWires j)
```

5 The view of the compiler

The execution interface is in `lean/Compiler.lean`. Let $\lambda = \lambda_{\mathsf{M}} + \lambda_{\mathsf{S}}$ and $K = 2^{2\lambda}$. A computation answer records both parties' matrix-phase and shortening-phase answers. A refresh or public-input sampler uses the same answer type in one simultaneous phase, with total per-party budget λ. Thus the full transcript has S answers, and the only additional visible data is the authorized plaintext tuple:

$$\mathsf{History} = \mathcal{Z}^{[S]}, \qquad \mathsf{View} = \mathcal{Z}^{[S]} \times \mathbb{F}_2^{[k]}.$$

No final share pair belongs to this view.

```
abbrev History (source : Source P U T k) (ml sl : ℕ) :=
  Composition.Answers (Compiler.Answer ml sl) source.operationCount

abbrev View (source : Source P U T k) (ml sl : ℕ) :=
  History source ml sl × (Fin k → Refreshing.Bit)
```

A deterministic strategy supplies local *callbacks*, the leakage functions evaluated during the matrix and rectangular-shortening samplers. Each callback can inspect only its active operation's permitted shares, messages, and complete private sampling record, including rejected proposals. All attempts within a phase share that phase's leakage budget. The callback domain does not contain unrelated retained wire encodings.

```
abbrev Strategy (_source : Source P U T k) (n ml sl : ℕ) :=
  CircuitCompiler.Operations.Strategy Refreshing.Bit
    (Refreshing.LeakageOutput ml) (Refreshing.LeakageOutput ml) (Refreshing.LeakageOutput sl) (
   ↪ Refreshing.LeakageOutput sl)
    n (shorteningExtra n) P (longDimension n)
```

The real distribution initializes the actual private inputs, executes every compiled instruction against the supplied strategy, and then applies the trusted decoder. The decoder reveals only the inner products at the designated original source-wire locations.

```
def runState {source : Source P U T k} {n ml sl : ℕ}
    (strategy : Strategy source n ml sl) (hn : 2≤n) (privateValues : Fin P → Refreshing.Bit) :=
  ((strategy.protocol hn).realRunner source.compiledProgram).execute
    (Sampling.Unbounded.toFinite (Sampling.RawEncoding.initializeShares hn privateValues))

def transcript {source : Source P U T k} {n ml sl : ℕ}
    (strategy : Strategy source n ml sl) (hn : 2≤n) (privateValues : Fin P → Refreshing.Bit) :
    Refresh.FiniteDistribution (History source ml sl) :=
  (runState strategy hn privateValues).map fun state =>
    Composition.vectorToAnswers _ _ state.2

def decode (source : Source P U T k) {n : ℕ}
    (wires : Fin (P+source.operationCount) → Refreshing.Share n × Refreshing.Share n) (j : Fin
   ↪ k) : Refreshing.Bit :=
  Refreshing.secret (wires (source.outputSlot j))

def realView {source : Source P U T k} {n ml sl : ℕ}
    (strategy : Strategy source n ml sl) (hn : 2≤n) (privateValues : Fin P → Refreshing.Bit) :
    Refresh.FiniteDistribution (View source ml sl) :=
  (runState strategy hn privateValues).map fun state =>
    (Composition.vectorToAnswers _ _ state.2,decode source state.1)
```

The simulated distribution executes the same program with zero private inputs, discards all its hidden pairs, and appends the supplied authorized output. It needs no input producing that output.

```
def simulatedView {source : Source P U T k} {n ml sl : ℕ}
    (strategy : Strategy source n ml sl) (hn : 2≤n) (authorizedOutput : Fin k → Refreshing.Bit)
   ↪  :
    Refresh.FiniteDistribution (View source ml sl) :=
  (transcript strategy hn (fun _=>0)).map fun history=>(history,authorizedOutput)
```

With $q = 2^n - 1$, the explicit bound is

$$10^6 \cdot S \cdot K^9 \cdot q^{-1/27}.$$

The direct-privacy proposition writes out this bound.

6 Local certificates and composition

The compiler proof separates the local estimates for individual operations from the global comparison of transcript distributions. The following interface records the bounds that composition uses for each approximated operation. The local interface is in `lean/Compiler.lean`. To explain it, fix a finite wire set W, S operations, a finite answer alphabet $\mathcal{Z}$, and scopes $V_i \subseteq W$ for every $i \in [S]$. For every formal preceding history h and answer $z \in \mathcal{Z}$, the true table $p_{i,h,z} : (U_n \times U_n)^W \to [0,1]$ depends only on V_i. Its answer weights sum to at most one; the headline theorem separately requires the complete transcript distributions to be normalized. The predicate $\mathsf{valid}(\sigma, i)$ states that the labeling $\sigma : W \to \mathbb{F}_2$ satisfies operation i's plaintext relation. Public-input samplers are outside the set approximated.

A chosen signed approximation has a representation

$$G(\ell, r) = \sum_j a_j(\ell) \cdot b_j(r), \qquad \Phi(\ell) = \sum_j a_j(\ell)^2, \qquad \Psi(r) = \sum_j b_j(r)^2.$$

Here $\ell, r \in U_n^W$ are the two parties' full share tuples. The *certificate*, a Lean record of the local approximation and its bounds, contains $\Phi \le 2/\beta$, $\Psi \le 2/\beta$, and separate uniform mean energies at most eight. For every valid labeling, its L_2 error under the original encoding distribution is at most ε. The formal local interface also records the all-label L_2 bound twelve. This uses the pre-shortening uniform L_2 bound of two for the doubled truncated table, the fixed-label density bound of 27, and contraction of the labeled L_2 norm under shortening: $2 \cdot \sqrt{27} \le 12$. The paper's analytical local bound instead uses the coarser nuclear-norm bound of four to obtain $C_0 = 4 \cdot \sqrt{27}$. The mean-energy bound eight is deliberately looser than the paper's analytical mean-energy bound of four; the larger formal constants are absorbed by the explicit 10^6 prefactor without changing the construction or the exponents. These are the fields of `GateLocalEstimates`; in this generic declaration, its type parameter `X` represents the full party tuple U_n^W.

```
structure GateLocalEstimates {X S : Type*} [Fintype X] [Nonempty X]
    (laws : S → FiniteDistribution (X × X)) (valid : S → Prop)
    (original : S → X → X → ℝ) (β ε : ℝ) where
  approximation : X → X → ℝ
  factors : Factors approximation
  left_max : ∀ x, factors.leftEnergy x ≤ 2 / β
  right_max : ∀ x, factors.rightEnergy x ≤ 2 / β
  left_mean : (FiniteDistribution.uniform : FiniteDistribution X).expect factors.leftEnergy ≤ 8
  right_mean : (FiniteDistribution.uniform : FiniteDistribution X).expect factors.rightEnergy ≤
   ↪ 8
  all_labels_rms : ∀ s, rms (laws s) (fun xy => approximation xy.1 xy.2) ≤ 12
  valid_error_rms : ∀ s, valid s →
    rms (laws s) (fun xy => original s xy.1 xy.2 - approximation xy.1 xy.2) ≤ ε
```

For a labeling σ, the unconditional encoding distribution is

$$\mu_\sigma = \bigotimes_{w \in W} \mu_{\sigma(w)}.$$

The function `groupedLaw` merely rearranges this sampled tuple into its two party coordinates, retaining each pair's correlation.

```
def groupedLaw (law : F → FiniteDistribution (X × X)) (label : W → F) :
    FiniteDistribution ((W → X) × (W → X)) :=
  groupedDistribution (fun w => law (label w)) Prod.fst Prod.snd
```

The record below chooses one approximation and one factor representation for every operation, formal history, and answer. The choice is shared by all labelings. Its locality fields ensure that separately trimming either party's energy still depends only on the operation's declared scope. No field assumes a comparison bound for a complete execution or a marked sum.

```
structure LocalData (scope : Fin S → Finset W)
    (network : ScopedProbabilities (E := X × X) (A := A) scope)
    (law : F → FiniteDistribution (X × X))
    (valid : (W → F) → Fin S → Prop) (approximated : Finset (Fin S)) (β ε : ℝ) where
  gate : ∀ i h z,
    GateLocalEstimates (groupedLaw law)
      (fun label => valid label i ∧ i ∈ approximated)
      (fun _ left right => network.prob i h (fun w => (left w,right w)) z) β ε
  left_locality : ∀ i h z j, Scoped (scope i)
    (fun e : W → X × X => (gate i h z).factors.left (fun w => (e w).1) j)
  right_locality : ∀ i h z j, Scoped (scope i)
    (fun e : W → X × X => (gate i h z).factors.right (fun w => (e w).2) j)
```

For a complete formal answer vector $z \in \mathcal{Z}^{[S]}$, the canonical weight is

$$P_\sigma(z) = \mathbb{E}_{e \leftarrow \mu_\sigma}\left[\prod_{i=0}^{S-1} p_{i,z_{<i},z_i}(e)\right].$$

This fixed-label interface samples the encoding tuple independently before any answer is generated. The causal interface below instead samples a wire when its label has been chosen. Neither formula replaces a conditional distribution after leakage by fresh independent encodings.

```
def historyWeight (scope : Fin S → Finset W)
    (network : ScopedProbabilities (E := X × X) (A := A) scope)
    (z : Fin S → A) (encoding : W → X × X) : ℝ :=
  ∏ i, network.prob i ((List.ofFn z).take i.val) encoding (z i)

def canonicalWeight (scope : Fin S → Finset W)
    (network : ScopedProbabilities (E := X × X) (A := A) scope)
    (law : F → FiniteDistribution (X × X)) (label : W → F) (z : Fin S → A) : ℝ :=
  (independentDistribution (fun w => law (label w))).expect
    (historyWeight scope network z)
```

The fixed-label component proposition is defined in `lean/Compiler.lean` and bundled in `Compiler.main_theorem`. It assumes $|V_i| \le 3$, incidence at most three, nonempty scopes for approximated operations, valid labelings, and agreement on public wires. Scopes of unapproximated operations must contain only public wires. Unused private inputs may differ because they are marginalized exactly. The local parameters are $\beta = q^{-1/3}$ and untrimmed error $10^4 \cdot q^{-1/9}$. The independent proof follows the paper's signed-composition argument: it trims the energies at $B = q^{1/9}$, proves the resulting L_3 estimate, applies Hölder's inequality for incidence three, tensors the signed encoding differences, and sums the marked terms before taking absolute values. The final comparison follows by finite inclusion–exclusion. These steps are checked in `lean/Proofs/Compiler/DirectComposition/`; the supporting proof terms are omitted here.

```
def directCompositionClaim.{u,v} : Prop :=
  ∀ {W : Type u} {A : Type v} [Fintype W] [DecidableEq W] [Fintype A] [Nonempty A] {S n : ℕ} [
   ↪ Nonempty (U Refreshing.Bit n)] (hn : 2≤n) (scope : Fin S→Finset W)
    (network : ScopedProbabilities (E := U Refreshing.Bit n×U Refreshing.Bit n) (A := A) scope)
    (valid : (W→Refreshing.Bit)→Fin S→Prop) (approximated : Finset (Fin S))
    (data : LocalData scope network (encodingDistribution hn) valid approximated
      (betaParameter (Fintype.card (U Refreshing.Bit n):ℝ)) (rawErrorParameter (Fintype.card (U
   ↪ Refreshing.Bit n):ℝ)))
    (hports : ∀ i,(scope i).card≤3)
    (hdegree : ∀ w,(Finset.univ.filter (fun i=>w∈scope i)).card≤3)
    (hnonempty : ∀ i∈approximated,(scope i).Nonempty)
    (publicWires : Finset W) (hpublic : ∀ i,i∉approximated→scope i⊆publicWires)
    (first second : W→Refreshing.Bit)
    (hfirst : ∀ i∈approximated,valid first i)
    (hsecond : ∀ i∈approximated,valid second i)
    (hlabels : ∀ w∈publicWires,first w=second w)
    (P Q : FiniteDistribution (Fin S→A))
    (hP : ∀ z,P.weight z=canonicalWeight scope network (encodingDistribution hn) first z)
    (hQ : ∀ z,Q.weight z=canonicalWeight scope network (encodingDistribution hn) second z),
    Refresh.statisticalDistance P Q≤1000000*(S:ℝ)*(Fintype.card A:ℝ)^9*
      (Fintype.card (U Refreshing.Bit n):ℝ)^(-1/27:ℝ)
```

7 Privacy of the compiler

The direct-privacy component proposition is defined in `lean/Compiler.lean` and bundled in `Compiler.main_theorem`. Its statement has no hypothesis about local certificates, canonical distributions, or incidence: the independent proof supplies them for the actual compiled instruction program and for the samplers with all their retries. For every admissible strategy, $n \geq 2$, and private input $x \in \mathbb{F}_2^P$, it compares the real view with the single dummy-input target supplied with $\Gamma(x)$.

```
def directPrivacyClaim : Prop :=
  ∀  {P U T k n ml sl : ℕ} (source : Source P U T k)
    (strategy : Strategy source n ml sl) (hn : 2≤n) (input : Fin P → Refreshing.Bit),
    Refresh.statisticalDistance (realView strategy hn input)
      (simulatedView strategy hn (source.outputValues input)) ≤
      1000000*(source.operationCount:ℝ)*
        (((2 ^ (2 * (ml + sl)) : ℕ) : ℝ))^9*
        (((2 ^ n - 1 : ℕ) : ℝ))^(-1/27:ℝ)
```

8 The counted simulator of the compiler

The interface and its shared probability and work definitions are in `lean/EfficientSimulation.lean`. For a view space $\mathcal{V}$, a counted run is a probability mass function on $\mathcal{V} \times \mathbb{N}$. Its coordinates are the visible result v and a private counter w. The counter records random sampling, scalar arithmetic and tests, and explicitly charged preparation operations, including every rejected proposal and retry. Routine memory access and address bookkeeping are left to the implementation analysis; adversary, leakage-function, and ideal-oracle evaluation are charged separately. The auxiliary-input space is $\mathcal{A}_{\mathrm{aux}} = \{0,1\}^*$.

```
abbrev AuxiliaryInput := List Bool
```

Projecting away the counter gives the visible marginal:

$$(\pi_1)_* P.$$

The `viewLaw` definition below packages this marginal as a finite distribution.

The expected counter is computed in the nonnegative extended reals:

$$\mathsf{expectedWork}(P) = \sum_{v,w} P(v,w) \cdot w \in [0,\infty].$$

Thus infinite additional expected work is not silently excluded.

```
def expectedWork {V : Type*} (run : PMF (V × ℕ)) : ℝ≥0∞ :=
  ∑' z, run z * (z.2 : ℝ≥0∞)
```

Charging R_{ev} per callback sends (v,w) to $(v, w + Q \cdot R_{\mathrm{ev}})$ without changing the visible marginal. For the compiler, the proved callback count is $Q = 8T + 2U$. The generic definition accepts any supplied number of calls.

```
def chargeOracleBudget {V : Type*} (run : PMF (V × ℕ)) (calls R : ℕ) : PMF (V × ℕ) :=
  run.map (fun z => (z.1, z.2 + calls * R))
```

A joint accounting distribution $\mathcal{Q}$ records $((v,w),a)$, where the additional work a may depend on the same hidden tapes as v and w. Correct accounting requires its first marginal to equal the counted run:

$$\mathsf{AccountingLaw}(\mathcal{V}) = \mathrm{PMF}((\mathcal{V} \times \mathbb{N}) \times \mathbb{N}), \qquad (\pi_1)_* \mathcal{Q} = P.$$

The map $((v,w),a) \mapsto (v, w+a)$ gives the total work. Its expectation is the expected counter value plus $\mathbb{E}_{\mathcal{Q}}[a]$, without any independence assumption.

```
abbrev AccountingLaw (V : Type*) := PMF ((V × ℕ) × ℕ)

def AccountsFor {V : Type*} (run : PMF (V × ℕ)) (accounting : AccountingLaw V) : Prop :=
  accounting.map Prod.fst = run

def totalRun {V : Type*} (accounting : AccountingLaw V) : PMF (V × ℕ) :=
  accounting.map (fun z => (z.1.1, z.1.2 + z.2))

def additionalExpectedWork {V : Type*} (accounting : AccountingLaw V) : ℝ≥0∞ :=
  ∑' z, accounting z * (z.2 : ℝ≥0∞)
```

For finite $\mathcal{V}$, the function `viewLaw` expresses the same visible marginal as a finite distribution with real masses. This generic conversion is defined in `lean/EfficientSimulation.lean` and is used by the simulator theorem below.

```
def viewLaw {V : Type*} [Fintype V] (run : PMF (V × ℕ)) : Refresh.FiniteDistribution V :=
  CircuitCompiler.Sampling.Unbounded.toFinite (run.map Prod.fst)
```

An adversary family chooses an admissible strategy from its auxiliary string, independently of the actual private input.

```
abbrev Adversary (source : Source P U T k) (n ml sl : ℕ) :=
  AuxiliaryInput → Compiler.Direct.Strategy source n ml sl
```

The counter charges P operations for preparing the zero private input array and k for copying the authorized output.

```
def preparation (P : ℕ) : Counted (Vector Refreshing.Bit P) :=
  tabulate fun _ => ⟨0,1⟩

def outputPreparation {k : ℕ} (output : Fin k → Refreshing.Bit) : Counted (Vector Refreshing.
    ↪ Bit k) :=
  tabulate fun j => ⟨output j,1⟩
```

The simulator takes only an auxiliary input and the authorized output, with the public source fixed in its type. Its return distribution records the visible view jointly with a private work counter:

$$\mathsf{Simulator} = \mathcal{A}_{\mathrm{aux}} \to \mathbb{F}_2^{[k]} \to \mathrm{PMF}(\mathsf{View} \times \mathbb{N}).$$

```
abbrev Simulator (source : Source P U T k) (n ml sl : ℕ) :=
  AuxiliaryInput → (Fin k → Refreshing.Bit) → PMF (Compiler.Direct.View source ml sl × ℕ)
```

This is the counted execution of the actual compiled program, including all matrix proposals and rectangular retries. It never evaluates the signed tables from the statistical proof.

```
def simulator (source : Source P U T k) {n ml sl : ℕ}
    (adversary : Adversary source n ml sl) (hn : 2≤n) : Simulator source n ml sl :=
  fun aux output =>
    let prepared := preparation P
    let reported := outputPreparation output
    (CircuitCompiler.Operations.Counted.execution source.compiledProgram (adversary aux) hn
      (fun i=>prepared.value.get i)).map fun z=>
        ((Composition.vectorToAnswers _ _ z.1.2,fun j=>reported.value.get j),
          prepared.operations+reported.operations+z.2)
```

Its explicit counter and callback budgets are

$$W = P + k + 16P \cdot (6n + 1) + 100S \cdot (n^2 + 2)^3,$$
$$Q = 8T + 2U.$$

Including the separate charge R_{ev} per callback gives the bound $W + Q \cdot R_{\mathrm{ev}}$; a bound on leaked answer length alone does not bound the evaluation time of arbitrary leakage functions. These formulas appear directly in the simulation proposition.

The direct-simulation component proposition is defined in `lean/EfficientSimulation.lean` and bundled in `EfficientSimulation.main_theorem`. It fixes the concrete simulator before the target-error parameter and the actual private input. For every prescribed output, its visible marginal is exactly the dummy-input target and the expected value of its counter is at most W. Additional work may depend on the same hidden tapes: the joint-accounting clause adds its expectation without an independence assumption, including infinite additional expectation. A single absolute constant $C \geq 1$ gives, for every $\kappa \geq 1$, distance at most $2^{-\kappa}$ whenever

$$n \geq C \cdot (\kappa + \lambda + \ln(S + 2)), \qquad n \geq 2.$$

The proved counterpart takes $C = 12000$. The exact marginal and work bounds hold at every $n \geq 2$, independently of this sufficient security threshold.

```
def directSimulationClaim : Prop :=
  ∃ C : ℝ, 1≤C ∧
      ∀ (n ml sl P U T k : ℕ) (hn : 2≤n)
        (source : Compiler.Direct.Source P U T k) (adversary : Adversary source n ml sl),
        ∃ S : Simulator source n ml sl,
          S=simulator source adversary hn ∧
          (∀ (aux : AuxiliaryInput) (y : Fin k → Refreshing.Bit),
            viewLaw (S aux y)=Compiler.Direct.simulatedView (adversary aux) hn y ∧
            expectedWork (S aux y)≤(P+k+P*(16*(6*n+1))+source.operationCount*(100*(n*n+2)^3):ℝ≥
   ↪ 0∞) ∧
            (∀ R : ℕ, expectedWork (chargeOracleBudget (S aux y) (8*T+2*U) R)≤
              (P+k+P*(16*(6*n+1))+source.operationCount*(100*(n*n+2)^3)+(8*T+2*U)*R:ℝ≥0∞)) ∧
            (∀ accounting : AccountingLaw (Compiler.Direct.View source ml sl),
              AccountsFor (S aux y) accounting→expectedWork (totalRun accounting)≤
                (P+k+P*(16*(6*n+1))+source.operationCount*(100*(n*n+2)^3):ℝ≥0∞)+
   ↪ additionalExpectedWork accounting)) ∧
          (∀ (κ : ℝ) (_hκ : 1≤κ) (aux : AuxiliaryInput) (input : Fin P → Refreshing.Bit),
            C*(κ+((ml+sl:ℕ):ℝ)+Real.log ((source.operationCount:ℝ)+2))≤(n:ℝ)→
            Refresh.statisticalDistance (Compiler.Direct.realView (adversary aux) hn input)
              (viewLaw (S aux (source.outputValues input)))≤Real.rpow 2 (-κ))
```

9 History-dependent labels and stateful circuits

The causal composition interface samples each encoding only after its plaintext label has been determined by earlier answers. The `Plan` and `Realization` definitions are in `lean/Compiler.lean`; their normalization proofs are in `lean/Proofs/Compiler/Adaptive/Model.lean`. A `Plan` starts with P initial encodings and appends one encoding and one answer per operation. Its new encoding distribution is selected using only earlier answers, before the current answer is sampled. The argument `answer` specifies the conditional answer distribution given the allocated encodings and the newly sampled encoding.

```
inductive Plan (E : Type u) (A : Type v) [Fintype E] [Fintype A] (P : ℕ) : ℕ → Type (max u v)
   ↪ where
  | nil : Plan E A P 0
  | snoc {S : ℕ} (prior : Plan E A P S)
      (output : (Fin S → A) → FiniteDistribution E)
      (answer : RunState E A P S → E → FiniteDistribution A) : Plan E A P (S+1)
```

A `Realization` relates this concrete forward experiment to the scoped probability functions and the labeling induced by each formal transcript. Its identities specify the wire distributions and the answer factors. They assume no privacy bound or marked-sum estimate. The wire equivalence permits arbitrary names for the chronological encoding locations.

```
structure Realization (network : ScopedProbabilities (E := E) (A := A) scope)
    (law : F → FiniteDistribution E) (label : (Fin S → A) → W → F) where
  inputs : ℕ
  addresses : Fin (inputs+S) ≃ W
  initial : Fin inputs → FiniteDistribution E
  plan : Plan E A inputs S
  wire_laws : ∀ z j, plan.wireLaws initial z j = law (label z (addresses j))
  answer_factors : ∀ e z i, plan.factors e z i =
    network.prob i ((List.ofFn z).take i.val) (fun w => e (addresses.symm w)) (z i)
```

The independent proof in `lean/Proofs/Compiler/DirectComposition/HistoryLabels.lean` sums outside encoding weights together with unguarded answers. This proves the normalization needed when future labels depend on earlier leakage. The adaptive-composition component proposition in `lean/Compiler.lean` keeps the same local certificates, incidence bounds, and quantitative error as the fixed-label proposition. Validity and agreement on public bits must hold at every formal transcript.

```
def adaptiveCompositionClaim.{u,v} : Prop :=
  ∀ {W : Type u} {A : Type v} [Fintype W] [DecidableEq W] [Fintype A] [Nonempty A] {S n : ℕ} [
   ↪ Nonempty (U Refreshing.Bit n)] (hn : 2 ≤ n) (scope : Fin S → Finset W)
    (network : ScopedProbabilities (E := U Refreshing.Bit n×U Refreshing.Bit n) (A := A) scope)
    (valid : (W → Refreshing.Bit) → Fin S → Prop) (approximated : Finset (Fin S))
    (data : LocalData scope network (encodingDistribution hn) valid approximated
      (betaParameter (Fintype.card (U Refreshing.Bit n):ℝ))
      (rawErrorParameter (Fintype.card (U Refreshing.Bit n):ℝ)))
    (hports : ∀ i, (scope i).card ≤ 3)
    (hdegree : ∀ w, (Finset.univ.filter (fun i => w ∈ scope i)).card ≤ 3)
    (hnonempty : ∀ i ∈ approximated, (scope i).Nonempty)
    (publicWires : Finset W) (hpublic : ∀ i, i ∉ approximated → scope i ⊆ publicWires)
    (first second : (Fin S → A) → W → Refreshing.Bit)
    (firstExecution : Compiler.Adaptive.Realization network (encodingDistribution hn) first)
    (secondExecution : Compiler.Adaptive.Realization network (encodingDistribution hn) second)
    (hfirst : ∀ z, ∀ i ∈ approximated, valid (first z) i)
    (hsecond : ∀ z, ∀ i ∈ approximated, valid (second z) i)
    (hlabels : ∀ z, ∀ w ∈ publicWires, first z w = second z w),
    Refresh.statisticalDistance firstExecution.transcript secondExecution.transcript ≤
      1000000*(S:ℝ)*(Fintype.card A:ℝ)^9*(Fintype.card (U Refreshing.Bit n):ℝ)^(-1/27:ℝ)
```

The stateful source in `lean/Compiler.lean` has m register bits, d public input bits, c constants, g NAND gates, and k output positions. Its designated wires specify the next register state and the public output. Repeated positions and directly designated input wires are allowed.

```
structure Circuit (stateBits inputBits constants gates outputBits : ℕ) where
  program : NandProgram (stateBits+inputBits+constants) gates
  constantValues : Fin constants → Refreshing.Bit
  nextState : Fin stateBits → Fin (stateBits+inputBits+constants+gates)
  outputs : Fin outputBits → Fin (stateBits+inputBits+constants+gates)
```

The public interaction is defined in `lean/Compiler.lean`. An input choice receives the invocation index, the leakage prefix at its start, and the earlier input/output records. The oracle interface receives the chronological request history and current request, and returns only the authorized plaintext output. A public history supplied to a leakage strategy contains only inputs already announced and outputs whose invocations have ended. The timing lemmas prove these availability

conditions, including for zero-operation transitions. A fixed auxiliary input and adversary tape are captured in these functions.

```
structure Adversary (m d k n ml sl : ℕ) where
  choose : InputChoice (Compiler.Answer ml sl) d k
  leakage : PublicHistory d k → LocalStrategy m n ml sl
```

The simulator encodes only an all-zero initial register state and executes the actual interleaved operations without revealing its dummy output values or encodings. Encodings needed for the next state or subsequent uses remain available internally and are inaccessible to unrelated leakage callbacks. They may be erased after their last required use; the mathematical view projection does not model physical erasure. Its public observation combines the leakage transcript with the causal input/output records obtained through the ideal oracle. The recursive records describe previously authorized responses that an online implementation can cache. The simulator has no argument for the actual initial register state.

```
def simulatedView {n ml sl : ℕ} (T : ℕ) (adversary : Adversary m d k n ml sl)
    (hn : 2 ≤ n) (oracle : Oracle d k) : FiniteDistribution (circuit.View T ml sl) :=
  (circuit.runState T adversary hn oracle (fun _ => 0)).map fun state =>
    circuit.observe T adversary oracle state.2
```

The independent stateful-privacy proof constructs the refresh chains across invocation boundaries, proves $S = 3Tg + Td + c$ and incidence three, and verifies the retained-state and trusted-decoder semantics. It then applies causal composition and the common public observation function to the real and zero-state transcripts. The reference explicitly chooses one simulator before quantifying the private initial state.

```
def statefulPrivacyClaim : Prop :=
  ∀  {m d c g k n ml sl : ℕ} (circuit : Circuit m d c g k)
    (T : ℕ) (adversary : Adversary m d k n ml sl) (hn : 2 ≤ n),
    ∃ simulate : Oracle d k → Refresh.FiniteDistribution (circuit.View T ml sl),
      simulate = circuit.simulatedView T adversary hn ∧
      ∀ initial : Fin m → Refreshing.Bit,
        Refresh.statisticalDistance (circuit.realView T adversary hn initial)
          (simulate (circuit.oracle initial)) ≤
          min 1 (1000000*(circuit.operationCount T:ℝ)*
            (((2 ^ (2 * (ml + sl)) : ℕ) : ℝ))^9*
            (((2 ^ n - 1 : ℕ) : ℝ))^(-1/27:ℝ))
```

The one admitted theorem in `lean/Compiler.lean` joins fixed-label composition, adaptive composition, direct privacy, and stateful privacy.

```
CircuitCompiler.AveragedEnergyComparison CompilerProof.DirectComposition
theorem main_theorem.{u,v} :
    Direct.directCompositionClaim.{u,v} ∧
    Direct.adaptiveCompositionClaim.{u,v} ∧
    Direct.directPrivacyClaim ∧ Stateful.statefulPrivacyClaim := by
  sorry
```

The stateful privacy proposition caps the bound by one:

$$\min\{1, 10^6 \cdot S \cdot K^9 \cdot q^{-1/27}\}.$$

The counted simulator additionally proves its exact visible marginal and, under the same cost convention, the expected counter bound

$$W = m + Tk + 16m \cdot (6n + 1) + 100S \cdot (n^2 + 2)^3,$$

and the exact callback count $8Tg + 2(Td + c)$. Its counter explicitly adds $m + Tk$ for preparing the zero state and reporting authorized outputs; unlike the direct simulator's tabulated preparations, this charge is added directly to the counted run. Ideal-oracle evaluation and adversary computations, including input choice, public-history processing, and leakage functions, are charged separately. The checked rate is $2 \cdot 10^6 \cdot S \cdot 2^{18\lambda - n/27}$, with the same conservative logarithmic threshold as above.

```
def statefulSimulationClaim : Prop :=
  ∀  {m d c g k n ml sl : ℕ} (circuit : Compiler.Stateful.Circuit m d c g k)
    (T : ℕ) (adversary : Compiler.Stateful.Adversary m d k n ml sl) (hn : 2 ≤ n),
    ∃ simulate : Compiler.Stateful.Oracle d k → PMF (circuit.View T ml sl × ℕ),
      simulate = simulator circuit T adversary hn ∧
      (∀ oracle,
        viewLaw (simulate oracle) = circuit.simulatedView T adversary hn oracle ∧
        expectedWork (simulate oracle) ≤ (m+T*k+(m*(16*(6*n+1))+(circuit.operationCount T)
  ↪ *(100*(n*n+2)^3)):ℝ≥0∞) ∧
        ∀ R : ℕ, expectedWork (chargeOracleBudget (simulate oracle) (circuit.callbackCount T) R)
  ↪  ≤
          (m+T*k+(m*(16*(6*n+1))+(circuit.operationCount T)*(100*(n*n+2)^3))+circuit.
  ↪ callbackCount T*R:ℝ≥0∞)) ∧
      ((1000000*(circuit.operationCount T:ℝ)*(((2 ^ (2 * (ml + sl)) : ℕ) : ℝ))^9*(((2 ^ n - 1 :
  ↪ ℕ) : ℝ))^(-1/27:ℝ)) ≤
        2000000*(circuit.operationCount T:ℝ)*(2:ℝ)^(18*((ml+sl:ℕ):ℝ)-(n:ℝ)/27)) ∧
      (∀ initial : Fin m → Refreshing.Bit,
        Refresh.statisticalDistance (circuit.realView T adversary hn initial)
          (viewLaw (simulate (circuit.oracle initial))) ≤ min 1 ((1000000*(circuit.
  ↪ operationCount T:ℝ)*(((2 ^ (2 * (ml + sl)) : ℕ) : ℝ))^9*(((2 ^ n - 1 : ℕ) : ℝ))^(-1/27:ℝ)
  ↪ )) ∧
        ∀ (κ : ℝ) (_hκ : 1 ≤ κ),
          12000*(κ+((ml+sl:ℕ):ℝ)+Real.log ((circuit.operationCount T:ℝ)+2)) ≤ n →
          Refresh.statisticalDistance (circuit.realView T adversary hn initial)
            (viewLaw (simulate (circuit.oracle initial))) ≤ (2:ℝ)^(-κ))
```

The one admitted theorem in `lean/EfficientSimulation.lean` joins the direct and stateful simulation guarantees.

```
theorem main_theorem : Direct.directSimulationClaim ∧ Stateful.statefulSimulationClaim := by
  sorry
```